

References:

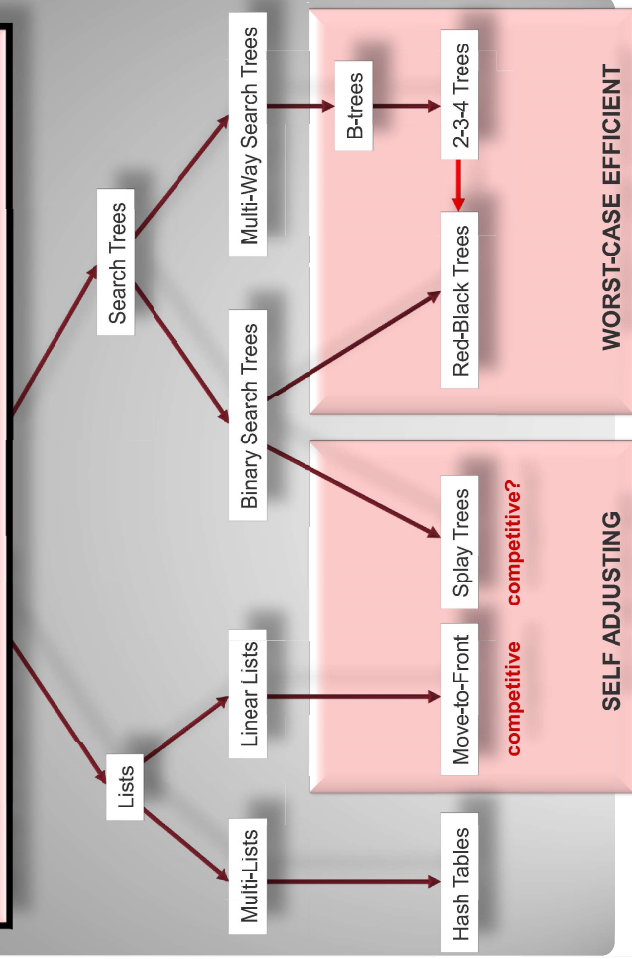
- Lecture Note 3
- AAW animation

Splay Tree: Self Adjusting BST

Self-Adjusting Binary Search trees

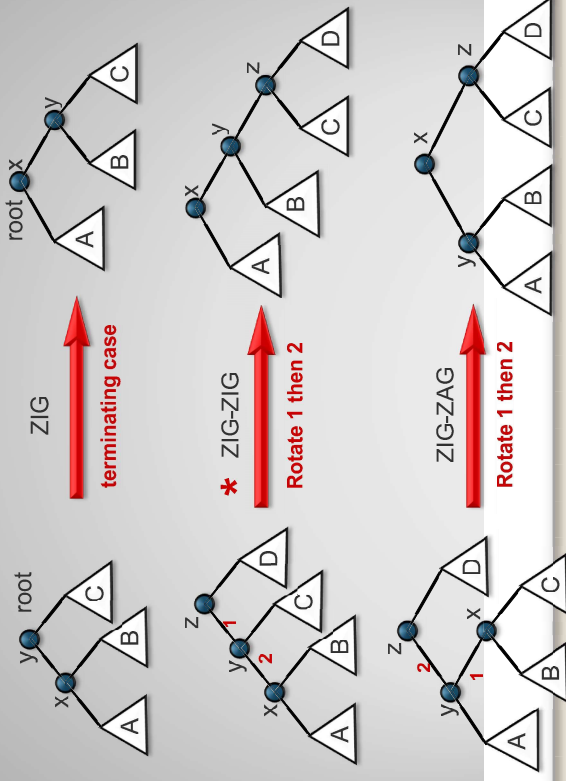
- **Red-Black trees** guarantee that each dictionary operation (Search, Insert, Delete) takes $O(\log n)$ time in the **worst-case**. They achieve this by enforcing an explicit “balance” constraint on the tree.
- Can we endow **self-adjusting** feature to the standard BST by the use of appropriate **rotations** to restructure the tree and improve the **efficiency** of **future operations** in order to keep the amortized cost per operation low? Is there a simple self-adjusting data structure that guarantees $O(\log n)$ amortized time per dictionary operation? **YES, Splay trees.**
- As with Red-Black trees, Splay trees give the following guarantee:
Any sequence of m dictionary operations on an initially empty dictionary takes in total $O(m \log n)$ time, where n is the max size of the dictionary during this sequence.
However, a single dictionary operation on Splay trees in the worst-case may cost up to $O(n)$ time.
- Splay Trees maintain a BST in an arbitrary state, with no “balance” information/constraint. However, they perform the simple, uniformly defined, **splay operation** after each dictionary operation. A splay operation on BSTs is analogous to **Move-to-Front** on linear lists.

DICTIONARIES



Splay(x)

Perform a series of the following splay steps, whichever applies, until x becomes the root of the tree. (Left-Right symmetric cases not shown.)



7

Generic Splay

- **GenericSplay(x):** Consider the links on the search path of node x in BST T . This operation performs one rotation per link on this path in some appropriate order to move node x up to the root of T .
- The number of rotations (at a cost of $O(1)$ per rotation) done by $\text{GenericSplay}(x)$ is $\text{depth}_T(x)$, i.e., depth of x in T before the splay. Access cost of node x is $O(1 + \text{depth}_T(x))$.
- **Generic Splay tree:** This is an arbitrary BST such that after each dictionary operation we perform $\text{GenericSplay}(x)$, where x is the deepest node accessed by the dictionary operation that is still in the tree. The cost of the dictionary operation, including the splay, is $O(1 + \text{depth}(x))$.
 - Which node is x ?
 - Successful search: x is the node just accessed.
 - Unsuccessful search: x is parent of the external node just accessed.
 - Insert: x is the node now holding the insertion key.
 - Unsuccessful Delete: same as unsuccessful search.
 - Successful Delete: x is parent of the spliced-out node.
- Any sequence of m dictionary operations takes $O(m+R)$ time, where R is the total number of rotations done by the generic splay operations in the sequence. So, we need to find an upper bound on R .

5

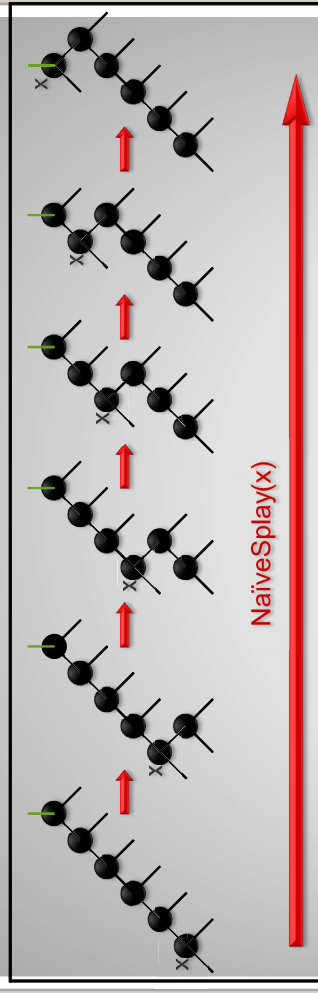
Splay Examples



8

Naïve Splay

NaïveSplay(x): A series of single rotations up the tree:
while $p[x] \neq \text{nil}$ **do** $\text{Rotate}(p[x], x)$



Exercise: Show that there is an adversarial sequence of n dictionary operations on an initially empty BST on which NaïveSplay takes $\Omega(n^2)$ time total, i.e., amortized $\Omega(n)$ time per dictionary operation.

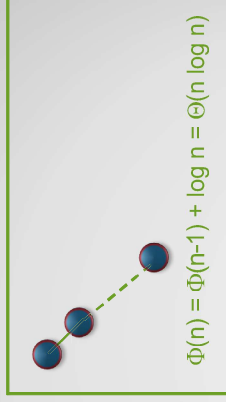
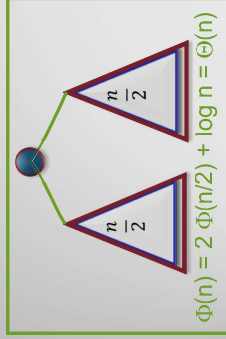
6

Potential Function (part 2/2)

Rank of x in T : $r_T(x) = \log w_T(x)$.

Potential: $\Phi(T) = \sum_{x \in T} r_T(x)$.

FACT: $\Phi(T)$ is also regular, and $\Theta(n) \leq \Phi(T) \leq \Theta(n \log n)$.



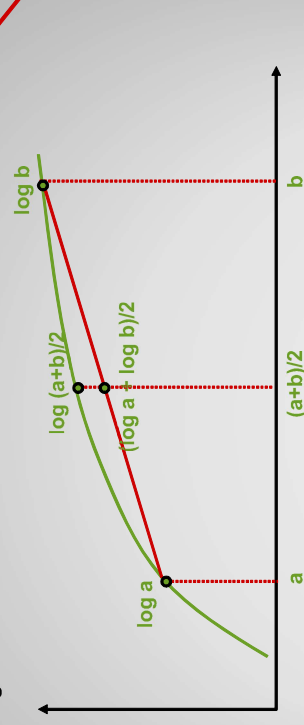
11

The LOG LEMMA

LEMMA 0: For any numbers $a, b, c \geq 1$
 $a + b \leq c \Rightarrow \log a + \log b \leq 2 \log c - 2$.

Proof 1: $c^2 \geq (a+b)^2 = 4ab + (a-b)^2 \geq 4ab$. So, $c^2 \geq 4ab$. Now take $\log$.

Proof 2: Logarithm is a concave function:



This makes the potential function work!

12

Amortized Analysis

The aggregate time on any sequence of m dictionary operations on an initially empty BST is

$O(m+R)$

R = total # rotations done by Splay operations in the sequence.

- We shall upper bound R by amortized analysis.
- We analyze amortized cost of Splay with unit cost per rotation.
- What is a "good" candidate for the potential function?

9

Potential Function (part 1/2)

$1 + \text{depth}_T(x)$ = # ancestors of x (including x) in T
 = access cost of x in T .

$w_T(x)$ = weight of x in T
 = # descendants of x (including x) in T
 = # nodes in the subtree rooted at x .

Candidate 1:

Internal path length of T : $L(T) = \sum_{x \in T} (1 + \text{depth}_T(x))$.

Candidate 2:

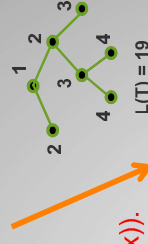
Weight of T : $W(T) = \sum_{x \in T} w_T(x)$.

FACT: (1) $\forall T, L(T) = W(T)$. Why?

(2) As potentials, $L(T)$ and $W(T)$ are regular (i.e., always ≥ 0 , $L(\emptyset) = W(\emptyset) = 0$).

(3) $\Theta(n \log n) \leq L(T) = W(T) \leq \Theta(n^2)$, where $n = |T|$.

$L(T)$ & $W(T)$ can become too large. They won't work! Dampen $W(T)$ by a sublinear function, e.g., $\log$.



10

ZIG-ZIG LEMMA



Proof:

$$\hat{c} = c + \Delta\Phi$$

$$= 2 + [r'(x) + r'(y) + r'(z)] - [r(x) + r(y) + r(z)]$$

$$\leq 2 + r'(x) + r'(y) + r'(z) - 2r(x) \quad \text{since } r'(x) = r(z), r'(y) \leq r'(x), r'(z) \geq r(x)$$

$$\leq 3(r'(x) - r(x)).$$

$$\text{since } w(x) + w'(z) \leq w'(x)$$

$$\Rightarrow r(x) + r'(z) \leq 2r'(x) - 2$$

by the LOG Lemma

$$\Rightarrow 2 + r'(z) \leq 2r'(x) - r(x)$$

15

ZIG LEMMA



Current
ranking of
node x and
y

$$r'(x)$$

$$r'(y)$$

After rotation
ranking of
node x and y

$$r'(x)$$

$$r'(y)$$

$$\text{and } r(x) \leq r'(x)$$

13

ZIG-ZAG LEMMA



Proof:

$$\hat{c} = c + \Delta\Phi$$

$$= 2 + [r'(x) + r'(y) + r'(z)] - [r(x) + r(y) + r(z)]$$

$$\leq 2 + r'(y) + r'(z) - 2r(x) \quad \text{since } r'(x) = r(z), r'(y) \geq r(x)$$

$$\leq 2(r'(x) - r(x))$$

$$\text{since } w'(y) + w'(z) \leq w'(x)$$

$$\Rightarrow r'(y) + r'(z) \leq 2r'(x) - 2$$

by the LOG Lemma

$$\Rightarrow 2 + r'(y) + r'(z) \leq 2r'(x)$$

$$\leq 3(r'(x) - r(x)).$$

16

ZIG LEMMA



Proof:

$$\hat{c} = c + \Delta\Phi$$

$$= 1 + [r'(x) + r'(y)] - [r(x) + r(y)]$$

$$= 1 + r'(y) - r(x) \quad \text{since } r'(x) = r(y)$$

$$\leq 1 + r'(x) - r(x) \quad \text{since } r'(y) \leq r'(x)$$

$$\leq 1 + 3[r'(x) - r(x)]. \quad \text{since } r(x) \leq r'(x)$$

14

Amortized cost of dictionary op's

LEMMA 4:

Let T be any n -node BST.

- (a) Increase in potential due to insertion of a new leaf into T , excluding the cost of splaying, is at most $\log n$.
- (b) Splicing-out any node from T cannot increase the potential.

Proof: Exercise 1.

THEOREM 2:

Aggregate number of rotations by any sequence of m dictionary operations on an initially empty Splay Tree of max size $\leq n$ is $R \leq m(4 \log n + 1)$.

Hence, aggregate execution time for such a sequence is at most $O(m+R) = O(m \log n)$.

Proof: Amortized number of rotations by a dictionary operation:

- Search: at most $1 + 3 \log n$ (by Theorem 1).
- Insert: at most $1 + 4 \log n$ (by Theorem 1 & Lemma 4a).
- Delete: at most $1 + 3 \log n$ (by Theorem 1 & Lemma 4b).

19

Lemmas on each Splay Step

LEMMA 1:



LEMMA 2:



LEMMA 3:



17

Amortized cost of Splay

THEOREM 1: Amortized number of rotations by a Splay operation on an n -node BST is at most $3 \log n + 1$.

Proof: Splay(x) operation is a sequence of splay steps:

$$\begin{aligned} T_{\text{before}} &= T_0 \xrightarrow{\dots} T_{i-1} \xrightarrow{c_i} T_i \xrightarrow{\dots} T_k = T_{\text{after}} \\ \Phi_{\text{before}} &= \Phi_0 \quad \hat{c}_i \quad \Phi_{i-1} \quad \Phi_i \quad \Phi_k = \Phi_{\text{after}} \end{aligned}$$

$$\hat{c} = c + \Phi_{\text{after}} - \Phi_{\text{before}}$$

$$= \sum_{i=1}^k (c_i + \Phi_i - \Phi_{i-1})$$

$$= \sum_{i=1}^k \hat{c}_i$$

$$\leq \sum_{i=1}^{k-1} 3(r_i(x) - r_{i-1}(x)) + (1 + 3(r_k(x) - r_{k-1}(x)))$$

$$= 1 + 3(r_{\text{after}}(x) - r_{\text{before}}(x))$$

$$\leq 1 + 3 \log n$$

$$\text{since } r_{\text{after}}(x) = \log n, r_{\text{before}}(x) \geq 0$$

18

Competitiveness?

20

Bibliography:

- [ST85] D.D. Sleator, R.E. Tarjan, "Self-adjusting binary search trees," J. ACM 32(3):652-686, 1985.
[Original paper on Splay Trees.]
- [Tar85] R.E. Tarjan, "Sequential access on splay trees takes linear time," Combinatorica 5(4), pp: 367-378, 1985.
- [DHKP09] E.D. Demaine, D. Harmon, J. Iacono, D. Kane, M. Patrascu, "The geometry of binary search trees," 20th ACM-SIAM Symposium on Discrete Algorithms (SODA), pp: 496-505, 2009.
- [WDS06] C.C. Wang, J. Derryberry, D.D. Sleator "O(lg lg n)-competitive dynamic binary search trees," 17th SODA, pp: 374-383, 2006.
- [Goo00] M.T. Goodrich, "Competitive tree-structured dictionaries," 11th SODA, pp:494-495, 2000.
- [CMSS00] R. Cole, B. Mishra, J. Schmidt, A. Siegel, "On the dynamic finger conjecture for splay trees. Parts I and II," SIAM J. on Computing 30(1), pp: 1-85, 2000.
- [Wei06] M.A. Weiss, "Data Structures and Algorithms in C++," 3rd edition, Addison Wesley, 2006.
- [GT02] M.T. Goodrich, R. Tamassia, "Algorithm Design: Foundations, Analysis, and Internet Examples," John Wiley & Sons, Inc., 2002.

23

Dynamic Optimality Conjecture

- Move-to-Front is competitive on linear lists with exchanges.
- Is SPLAY competitive on binary search trees with rotations?
- Sleator, Tarjan, "Self-adjusting binary search trees," J. ACM 32(3), pp: 652-686, 1985

originally proposed Splay Trees and made the following bold conjecture:

Dynamic Optimality Conjecture:

s = any sequence of dictionary operations starting with any n -node BST.

OPT = optimum off-line algorithm on s .

OPT carries out each operation by starting from the root and accessing nodes on the corresponding search path, and possibly some additional nodes, at a cost of one per accessed node, and that between each operation performs an arbitrary number of rotations on any accessed node in the tree, at a cost of one per rotation.

Then, $C_{\text{SPLAY}}(s) = O(n + C_{\text{OPT}}(s))$.

21

Special Cases

- Dynamic Optimality Conjecture is a very strong claim and is still open.
- Some special cases have been confirmed.
- The following paper gives a survey, offers alternatives to splaying, and develops a new method to aid finding competitive online BST algorithms:

Demaine, Harmon, Iacono, Kane, Patrascu,

"The geometry of binary search trees," SODA, pp: 496-505, 2009.

[To see this paper, click [here](#).]

- Some known special cases:

- **Sequential Access [Tar85]:** Access each node of a given n -node BST in order (smallest, then 2nd smallest, ..., then largest) always starting an access from the root. This takes $\Theta(n)$ time on Splay trees! Red-Black trees are not competitive and take $\Theta(n \log n)$ time.
- **Static Optimality [ST85]:** Access only operations on an n -node BST. Static OPT (no rotations allowed) [Knuth's dynamic programming algorithm]. (See AAW animation on Optimal Static BST, my EECS 3101 LS7 or LN9.) Red-Black trees are not competitive.
- **Dynamic Finger Search Optimality [CMSS00].**
- **Working-set bound [ST85].**
- **Entropy bound [ST85].**

24

Exercises

8. [Goodrich-Tamassia, p. 676] Energy-Balanced Binary Search Tree (EB-BST):

Red-Black trees use rotations to explicitly maintain a balanced BST to ensure efficiency in the dictionary operations. Splay trees are a self-adjusting alternative. On the other hand, an **Energy-Balanced BST** explicitly stores a **potential energy** parameter at each node in the tree. As dictionary operations are performed, the potential energies of tree nodes are increased or decreased. Whenever the potential energy of a node reaches a **threshold** level, we **rebuild** the subtree rooted at that node.

More specifically, an EB-BST is a binary search tree T in which each node x maintains $w[x]$ (weight of x , i.e., the number of nodes in the subtree rooted at x , including x . Assume $w[nil]=0$). More importantly, each node x of T also maintains a potential energy parameter $p[x]$.

Search, insert and delete operations are done as in standard (unbalanced) binary search trees, with one small modification. Every time we perform an insert or delete which traverses a search path from the root of T to a node v in T , we increment $p[x]$ by 1 for each node x on that search path. If there is no node on this path such that $p[x] \geq w[x]/2$, then we are done. Otherwise, let x be the highest node in T (i.e., closest to the root) such that $p[x] \geq w[x]/2$. We rebuild the subtree rooted at x as a completely balanced binary search tree, and we zero out the potential fields of each node in this subtree (including x).

(Note: a **binary tree** is called **completely balanced**, if for each node, the sizes of the left and right subtrees of that node differ by at most one.)

(a) Show the rebuilding can be done in time linear in the size of the subtree that is being rebuilt. That is, design and analyze an algorithm that converts an arbitrary given BST into an equivalent completely balanced BST in time linear in the size of that tree.

(b) Prove that if x and y are any two sibling nodes in an EB-BST, then $w[y] \leq 3w[x] + 2$.

[Hint: observe the potential energy of their parent node.]

(c) Using part (b), prove that the maximum height of any n -node EB-BST is $O(\log n)$.

(d) Prove that the amortized time of any dictionary operation on any n -node EB-BST is $O(\log n)$.

27

END

28

1. Prove Lemma 4.

2. Consider the sequence of $O(n)$ dictionary operations performed by procedure **Test** below.

procedure Test(n)

$T \leftarrow \emptyset$ (* T is initialized to an empty dictionary *)

for $i \leftarrow 1..n$ **do** Insert(i, T)

for $i \leftarrow 1..n$ **do** Search(i, T)

end

Show the following time complexities hold for procedure **Test**:

(a) $\Theta(n \log n)$ time (i.e., at least $\Omega(n \log n)$ and at most $O(n \log n)$ time) if T is a Red-Black tree.

(b) At least $\Omega(n^2)$ time if T is a NaïveSplay tree.

(c) At most $O(n \log n)$ time if T is a Splay tree.

(d) [Hard: Extra Credit and Optional] Only $\Theta(n)$ time if T is a Splay tree.

3. For any positive integers m and n , $n \leq m$, let $\Sigma(m, n)$ be the set of all possible sequences s of m dictionary operations applied to an initially empty dictionary, where n is the maximum number of keys in the dictionary at any time during s . We have already seen that executing any sequence $s \in \Sigma(m, n)$ takes at most $O(m \log n)$ time on both Red-Black trees and Splay trees. On the other hand, Splay trees tend to adjust to the pattern of usage. Prove for all m and n as above, there is a sequence $s \in \Sigma(m, n)$ whose execution takes at least $\Omega(m \log n)$ time on Red-Black trees, but at most $O(m)$ time on Splay trees.

4. Consider inserting the n keys $(1, 2, \dots, n)$, in that order, into an initially empty search tree T .

(a) What is the tight asymptotic running time of this insertion sequence if T is a Red-Black tree?

Describe the structure of the final Red-Black tree as accurately as you can.

(b) Answer the same questions as in (a) if T is a Splay tree.

(c) Consider a worst permutation of the n keys $(1, 2, \dots, n)$, so that if these n keys are inserted into an initially empty Splay tree, in the order given by that permutation, the running time is asymptotically maximized.

Characterize one such worst permutation for Splay trees. What would be the tight asymptotic running time for the insertion sequence according to that permutation?

5. **Split and Join on Splay trees:** These are cut and paste operations on dictionaries. The Split operation

takes as input a dictionary (a set of keys) A and a key value K (not necessarily in A), and splits A into two disjoint dictionaries $B = \{x \in A \mid \text{key}[x] \leq K\}$ and $C = \{x \in A \mid \text{key}[x] > K\}$. (Dictionary A is destroyed as a result of this operation.) The Join operation is essentially the reverse; it takes two input dictionaries A and B such that every key in $A <$ every key in B , and replaces them with their union dictionary $C = A \cup B$. (A and B are destroyed as a result of this operation.) Design efficient Split and Join on Splay trees and do their amortized analysis.

[Note: Split and Join, as defined here, is the same we gave on BST's, 2-3-4 trees, and Red-Black trees.]

6. **Top-Down Splay:** Describe how to do top-down splay and analyze its amortized cost.

7. [Goodrich-Tamassia, C-3.23, C-3.24, p. 215] Let T be a Splay Tree consisting of the n sorted keys $\langle x_1, x_2, \dots, x_n \rangle$. Consider an arbitrary sequence s of m successful searches on items in T . Let f_i be the access frequency of x_i in s . Assume each item is searched at least once (i.e., $f_i \geq 1$ for each $i=1..n$). (a) Show that the total execution time for sequence s on the Splay tree starting with T is

$$O(m + \sum_{i=1}^n f_i \log \frac{m}{f_i}).$$

[Hint: Modify the potential function by redefining the weight of a node as the sum of access frequencies of its descendants including itself.]

(b) Design and analyze an algorithm that constructs an off-line static (i.e., fixed) Binary Search Tree T' consisting of the same items $\langle x_1, x_2, \dots, x_n \rangle$, such that depth of item x_i in T' is $O(\log(m/f_i))$. Your algorithm should take at most $O(n \log n)$ time, and for an extra credit, it should take $O(n)$ time.

[Hint: Let x_i be the root, where i is the smallest index such that $f_1 + f_2 + \dots + f_i \geq m/2$. Recurse on that idea over the subtrees.]

(c) How do parts (a) and (b) above compare the execution times over the access sequence s on the on-line self-adjusting Splay tree T versus the off-line static binary search tree T' ?

[Hint: You may also consult [CLRS §15.5], AAW, and my EEC3101 Lecture Note 9 and Slide 7 on Knuth's dynamic programming algorithm that constructs the optimal static binary search tree.]

28